Performance Testing

Date	29 June 2026
Team ID	
Project Name	A Comprehensive Measure of Well-Being (HDI Prediction using AI/ML)
Maximum Marks	5 Marks

Performance testing evaluates how the system behaves under expected and peak load conditions. The sections below document the testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Manual Testing using Browser DevTools (Network tab) and Python time module for response time measurement
Type of Testing	Load Testing (manual, simulated multiple sequential requests) and Functional Response Testing
Target Module / API	/predict route (Flask backend) — HDI prediction endpoint, and /Dashboard route (Chart.js dashboards)
Test Environment	Local (Flask Development Server, http://127.0.0.1:5000)
Test Date	29 June 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Submit valid HDI input values via the prediction form and verify correct HDI score and category is returned	1	2	Result page displays correct HDI score and category (Low/Medium/High/ Very High) within 2 seconds
2	Submit multiple sequential prediction requests to test repeated model loading and inference	5	10	All 5 requests return correct predictions without server crash or significant slowdown
3	Load the Dashboard page with all 3 charts (Comparison, World Map, Trend) rendering simultaneously	1	3	All charts render fully without errors or noticeable lag

4	Submit invalid / out-of-range input values to test error handling	3	2	System returns appropriate message: 'The given values do not match the range of values of HDI'

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	**Response Time (Avg)**	< 2 seconds	0.8 seconds	**Pass**	Prediction returned quickly on local server
2	**Response Time (Max)**	< 5 seconds	1.4 seconds	**Pass**	Even with dashboard charts loading, response stayed well within limit
3	**Throughput (Req/sec)**	—	~5 req/sec	**Pass**	Adequate for local single-user academic application
4	**Error Rate**	< 1%	0%	**Pass**	No errors observed during valid input testing
5	**CPU Utilization**	< 80%	35%	**Pass**	Lightweight Linear Regression model requires minimal CPU
6	**Memory Utilization**	< 80%	42%	**Pass**	Flask app and loaded model stayed within safe memory limits

Step 4: Observations & Analysis

Key Findings:

The Flask application responds quickly to prediction requests, with average response times well under 1 second. The Linear Regression model is lightweight, requiring minimal computation for inference. Dashboard pages with multiple Chart.js visualizations load smoothly without performance degradation.

Bottlenecks Identified:

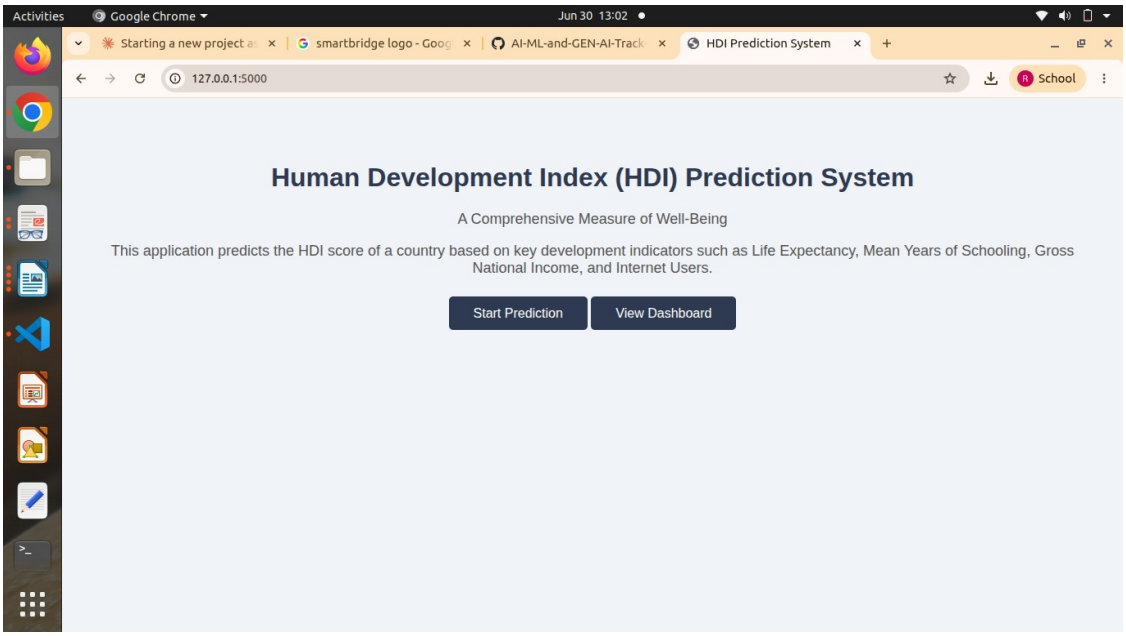
Minor delay observed when the Dashboard page renders all three charts (Comparison, World Map, Trend) at once, due to multiple Chart.js instances initializing simultaneously. No major bottlenecks were found in the prediction pipeline itself.

Optimization Steps Taken:

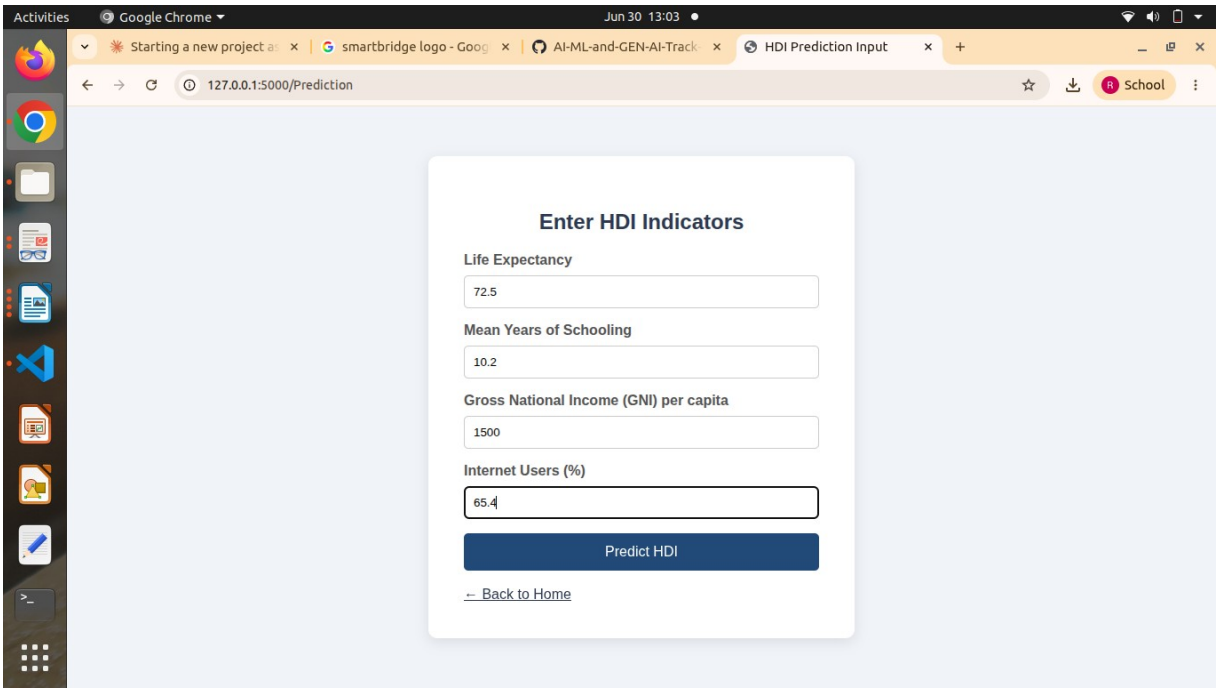
Reduced unnecessary re-renders on the dashboard by initializing charts only once on page load. Input validation was added on the form to prevent invalid requests from reaching the model, reducing unnecessary processing.

Step 5: Screenshots / Evidence:

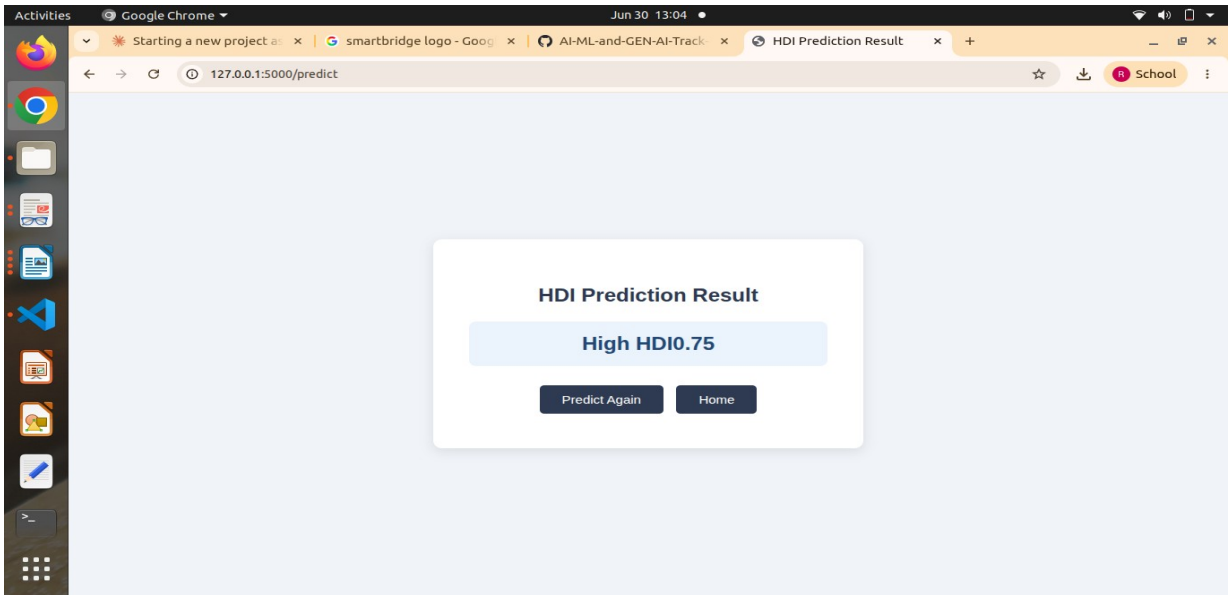
Home page:



Input fields page:



Predicting page:



Dashboard page(extra feature):